MÓDULO 2 — DEEP LEARNING

Clase de hoy

IA en acción: 3 demos en vivo

Del código a la realidad — sin instalaciones, solo resultados.

01

MNIST Canvas

02

YOLO Objetos

03

Chat con LLM

Agenda — clase de hoy



👋 Bienvenida + contexto

¿Por qué ver demos? Conectamos con lo que ya saben del módulo.



🖍️ Demo 01 — MNIST Canvas

Entrenar una red neuronal en Colab y clasificar dígitos dibujados a mano.



👁️ Demo 02 — YOLO: detección de objetos

Ver una app de detección en tiempo real y entender cómo está construida.



💬 Demo 03 — Chat con LLM local

App web conectada a un modelo LLM corriendo en tu máquina con Ollama.



🎯 Cierre + reflexión

¿Qué tienen en común estas 3 demos? Próximos pasos del módulo.

¿Por qué ver demos hoy?

Todo lo que vieron en las clases anteriores existe aquí:

vimos en clase

Neurona artificial + perceptrón → MNIST: cada píxel es una entrada, la red aprende a distinguir trazos

vimos en clase

Arquitecturas profundas (capas) → YOLO: una CNN de 9 millones de parámetros corriendo en tiempo real

vimos en clase

Modelos de lenguaje (LLMs) → Chat: un LLM corriendo localmente en tu computadora, sin internet

La teoría sin práctica es mapa sin territorio. Hoy vemos el territorio.

01

MNIST Canvas — Dibujá un número, la red lo reconoce

Lo que van a ver en esta demo:

1

El dataset MNIST

70.000 imágenes de dígitos escritos a mano. Cada imagen = 784 pixels (28 x 28)

2

Entrenamiento en vivo

Una red neuronal densa (784→128→64→10) entrenando en Colab. Se ve el accuracy subir época a época.

3

El momento estelar

Dibujás un número con el mouse. La red predice en tiempo real con una barra de confianza para cada dígito.

4

Los límites del modelo

Rotamos el dígito 90° y el modelo falla. ¿Por qué? Esto motiva las CNNs (Módulo 3).

🛠️ Herramientas: Google Colab · TensorFlow/Keras · Python · ipywidgets

01

La arquitectura de la red — 3 capas, 109.000 parámetros

ENTRADA	OCULTA 1	OCULTA 2	SALIDA
784 neuronas (28×28 píxeles)	128 neuronas ReLU + Dropout	64 neuronas ReLU + Dropout	10 neuronas Softmax = fórmula para calcular la probabilidad de una clase
0 params (entrada)	**100.480 params** 784×128 + 128	**8.256 params** 128×64 + 64	**650 params** 64×10 + 10

TOTAL

109.386

parámetros entrenables

(pesos + sesgos)

~98% accuracy en test set

¿Qué pasa cuando dibujás un 7?

Tu trazo (256×256 px) → Escalar a 28×28 → Aplanar 784 números → Forward pass → Softmax → 10 probs → Resultado: pred=7, 96%

02

YOLO v8n — Detección de objetos en tiempo real

¿Qué es YOLO?

YOLO = **You Only Look Once**. Una red neuronal que analiza la imagen UNA sola vez y detecta todos los objetos de golpe.

¿Qué detecta?

80 clases de objetos: personas, autos, perros, teléfonos, sillas, computadoras… lo que el modelo fue entrenado con COCO dataset.

¿Qué tan rápido?

La versión 'nano' (v8n) corre a 30+ FPS en una CPU normal. Sin GPU. En tiempo real.

¿Cómo funciona?

Divide la imagen en una grilla. Para cada celda predice: ¿hay un objeto? ¿dónde? ¿qué es? Todo en un solo forward pass.

¿Cómo está hecha?

Python + ultralytics + OpenCV. La app ya está corriendo — la van a ver en vivo y entender parte del código.

🛠️ Herramientas: Python · ultralytics · OpenCV · YOLOv8n (modelo pre-entrenado en COCO)

02

Las 4 líneas clave de la app YOLO

La app completa son menos de 100 líneas de Python. Estas son las 4 que importan:

1 Cargar el modelo

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
```

Una sola línea descarga y carga el modelo pre-entrenado (6MB).
No entrenamos nada — usamos Transfer Learning.

2 Leer la cámara

```
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
```

OpenCV abre la cámara y captura un frame. Frame = imagen numpy de (480, 640, 3).

3 Predecir

```
results = model(frame)
boxes = results[0].boxes
```

Forward pass sobre el frame completo.
Devuelve coordenadas de cada objeto detectado + clase + confianza.

4 Dibujar los resultados

```
annotated = results[0].plot()
cv2.imshow('YOLO', annotated)
```

Dibuja los bounding boxes con etiquetas y muestra el resultado.
Esto se repite 30 veces por segundo.

03

Chat con LLM local — Ollama + app web

¿Qué van a ver?

Una app web tipo ChatGPT, corriendo completamente en una computadora local, sin mandar datos a internet. El LLM corre con Ollama y la interfaz fue construida con Lovable.dev usando un prompt generado por Gemini.

¿Qué es Ollama?

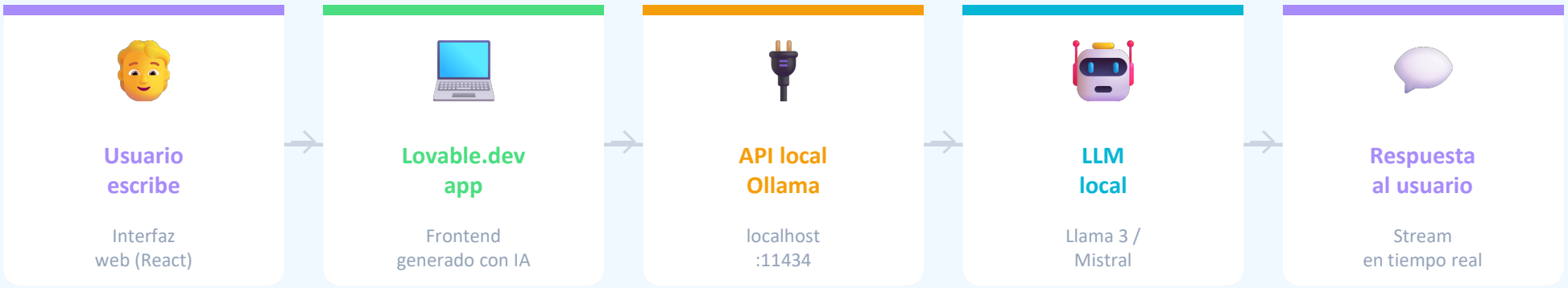
- Herramienta open source para correr LLMs en tu computadora
- Un comando y el modelo se descarga y queda listo
- Expone una API local en localhost:11434
- Compatible con Llama 3, Mistral, Gemma, Phi y más

¿Qué es Lovable.dev?

- Generador de apps web completas con IA
- Le das un prompt y genera React + backend
- El prompt fue diseñado con Gemini para ser preciso
- El resultado: interfaz de chat funcional en minutos

03

Cómo está construida — el flujo completo



Los 3 conceptos clave que van a aprender viendo esta app:

1. LLMs locales son posibles:
2. El prompt importa tanto como el modelo:
3. La IA como asistente de desarrollo:

¿Qué tienen en común estas 3 demos?



Bloques que ya conocés

Cada app usa capas densas, pesos, activaciones, loss functions — exactamente lo que estamos viendo en el módulo.



Modelos pre-entrenados

Ninguna entrenó desde cero en producción. YOLO usa COCO, el chat usa Llama 3. Transfer Learning en acción.



Herramientas abiertas

Python, Keras, ultralytics, Ollama: todo open source y gratuito. La IA de frontera ya es accesible.



IA construida con IA

El prompt de Lovable fue generado con Gemini. La IA ya se usa para construir IA. Esto cambia el rol del programador, “no lo reemplaza”.

Próxima clase: Regularización — Dropout, Batch Norm y Early Stopping →